

# LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

**Section:** 3. Python Data Types

## Lecture: 25. Type Conversion

### Section 1: Lecture Summary

This lecture focuses on Python's built-in **type conversion functions**, which enable the conversion of data from one type to another. The main functions covered are `int()`, `float()`, `bool()`, `complex()`, and `str()`. Each function's behavior is explained in detail, including which data types it can convert, how it handles different inputs (valid or invalid), and exceptions to their conversion abilities (such as when conversions fail or are not supported). The lecture includes practical demonstrations using examples in Jupyter Notebook, showing how these functions interact with different Python data types such as integers, floats, booleans, complex numbers, and strings. Key aspects include how decimals are truncated in integer conversion from floats, the representation of booleans as integers and floats, conversion of string literals in various numeral bases to integers, and how complex numbers are represented. The lecture emphasizes visualizing memory storage of variables and encourages hands-on practice.

### Section 2: Key Concepts and Explanations

#### 1. Type Conversion Functions in Python

- Functions that convert data from one type to another: `int()`, `float()`, `bool()`, `complex()`, `str()`
- These functions take an input (argument) and return the value converted to the specified type.

#### 2. Data Types Covered

- Integer
- Float

- Boolean
- Complex
- String

### 3. `int()` Function

- Converts various data types to integer.
- **Supported inputs:** float, boolean, valid numeric strings, but **not complex**.
- When converting float to int: decimal part is truncated (not rounded). E.g., `int(16.59)` → `16`
- Boolean: `True` → `1`, `False` → `0`
- Strings must represent a valid integer literal to convert successfully, e.g., `'125'` → `125`
- Conversion from strings with numeral prefixes requires specifying base:
  - Binary string: `'0b1010'` → base 2 → `10`
  - Hexadecimal string: `'0xA'` → base 16 → `10`
- Passing a non-valid string (e.g., `'Alexa'`) or complex numbers results in an error.

### 4. `float()` Function

- Converts compatible data types to float.
- **Supported inputs:** integer, boolean, valid numeric strings, but **not complex** (raises error).
- Integer converts by adding `.0`. E.g., `float(125)` → `125.0`
- Boolean: `True` → `1.0`, `False` → `0.0`
- String must be a valid float literal, e.g., `'12.75'` → `12.75` as float.

### 5. `bool()` Function

- Converts any data type to a boolean (`True` or `False`).
- Always returns `True` **except** in these three cases:

- The value `False` (boolean constant)
- Numeric zero (`0` or `0.0`)
- Empty or "empty-like" values such as empty string `""` or no argument passed
- Examples:
  - `bool(10) → True`
  - `bool(-12) → True`
  - `bool(0) → False`
  - `bool('False') (string) → True`
  - `bool(False) (boolean constant) → False`

## 6. `complex()` Function

- Converts data types into complex numbers.
- **Supported inputs:** integer, float, boolean, valid complex string literal, string representation of complex numbers.
- Converts real inputs as real part; imaginary part as `0j` by default.
- E.g., `complex(10) → (10+0j)`, `complex(-12.5) → (-12.5+0j)`
- Boolean: `True → (1+0j)`, `False → 0j`
- String must be a valid complex number literal like `'3+4j'`.

## 7. `str()` Function

- Converts any input to its string representation.
- Converts all data types (integer, float, boolean, complex, string) to strings without exceptions.
- E.g., `str(10) → '10'`, `str(-12.5) → '-12.5'`, `str(True) → 'True'`, `str(3+4j) → '3+4j'`.

## Section 3: Example Code and Use Cases

### `int()` examples

Float to int conversion truncates decimal part

Boolean to int conversion: True -> 1

Valid numeric string to int conversion

Binary string to int (base 2)

Hexadecimal string to int (base 16)

### `float()` examples

Integer to float conversion

Boolean to float conversion

Valid numeric string to float conversion

### `bool()` examples

### `complex()` examples

### `str()` examples

```
f = 16.59
b = True
s1 = '125'
s2 = '0b1010' # binary string
s3 = '0xA'    # hexadecimal string

x = int(f)
print(x, type(x)) # Output: 16 <class 'int'>

x = int(b)
print(x, type(x)) # Output: 1 <class 'int'>

x = int(s1)
print(x, type(x)) # Output: 125 <class 'int'>

x = int(s2, 2)
print(x, type(x)) # Output: 10 <class 'int'>

x = int(s3, 16)
print(x, type(x)) # Output: 10 <class 'int'>
```



```
i = 125
b = True
s = '12.75'

x = float(i)
print(x, type(x)) # Output: 125.0 <class 'float'>

x = float(b)
print(x, type(x)) # Output: 1.0 <class 'float'>

x = float(s)
print(x, type(x)) # Output: 12.75 <class 'float'>


i1 = 10
i2 = -12
f = -1.2e-3
c = 3+4j
s = 'False'

print(bool(i1)) # True
print(bool(i2)) # True
print(bool(f)) # True
print(bool(c)) # True
print(bool(s)) # True (string is non-empty)
print(bool(False)) # False (boolean constant False)
print(bool(0)) # False (numeric zero)
print(bool('')) # False (empty string)
print(bool()) # False (no argument defaults to False)


i = 10
f = 12.5
b1 = True
b2 = False
s = '3+4j'

print(complex(i)) # (10+0j)
print(complex(-12.5)) # (-12.5+0j)
print(complex(b1)) # (1+0j)
print(complex(b2)) # 0j
print(complex(s)) # (3+4j)


i1 = 10
```

```
i2 = -12
f = 1.2e-3
b1 = False
c = 3+4j

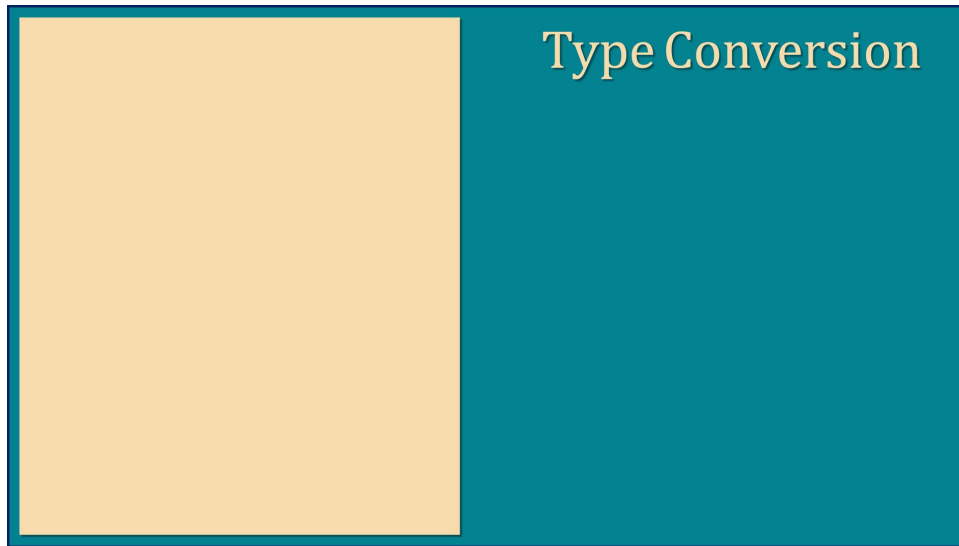
print(str(i1)) # '10'
print(str(i2)) # '-12'
print(str(f))  # '0.0012'
print(str(b1)) # 'False'
print(str(c))  # '3+4j'
```

## Section 4: Key Takeaways

- Python provides built-in functions for **type conversion**: `int()`, `float()`, `bool()`, `complex()`, and `str()`.
- `int()` converts float (truncates decimals), boolean (`True`→1, `False`→0), and valid numeric strings to integers; it cannot convert complex numbers or invalid strings without error.
- `float()` converts integers, booleans, and valid numeric strings to floats; it does not convert complex numbers.
- `bool()` converts all data types to boolean values, mostly returning `True`, except for specific "falsey" values: `False`, `0`, empty strings, or no input.
- `complex()` converts integers, floats, booleans, and valid complex number strings into complex numbers, placing the input as the real part and zero as imaginary if not specified.
- `str()` converts any data type to its string representation without errors.
- When converting strings representing numbers in different bases (binary, octal, hexadecimal), you must specify the base explicitly in `int()`.
- Understanding these conversions helps in managing data types and avoiding errors during programming in Python.
- Familiarity with these functions is foundational for effective Python programming.

## Lecture in Slides

**Please Note:** This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Type Conversion

**Functions**

Type Conversion

**Functions**

`int()`

## Type Conversion

### Functions

`int()`

`float()`

## Type Conversion

### Functions

`int()`

`float()`

`bool()`

Type Conversion

Functions

int()

float()

bool()

complex()

Type Conversion

Functions

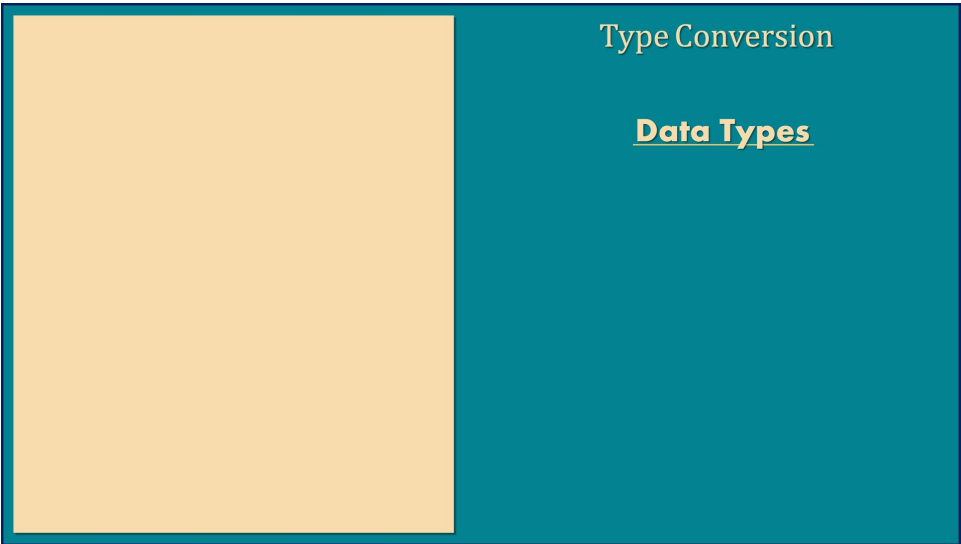
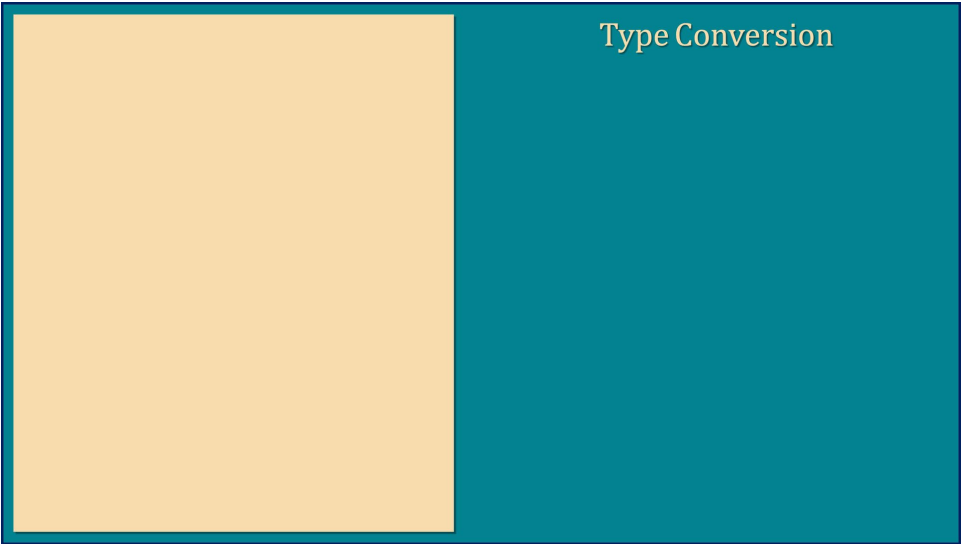
int()

float()

bool()

complex()

str()



Type Conversion  
Data Types  
Integer

Type Conversion  
Data Types  
Integer  
Float



## Type Conversion

### Data Types

Integer

Float

Boolean

Type Conversion

Data Types

Integer

Float

Boolean

Complex

Type Conversion

Data Types

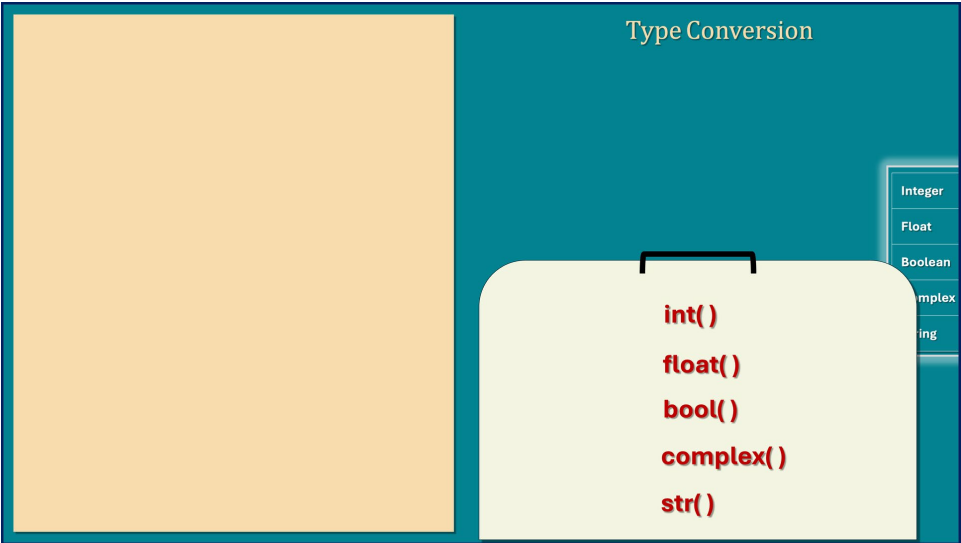
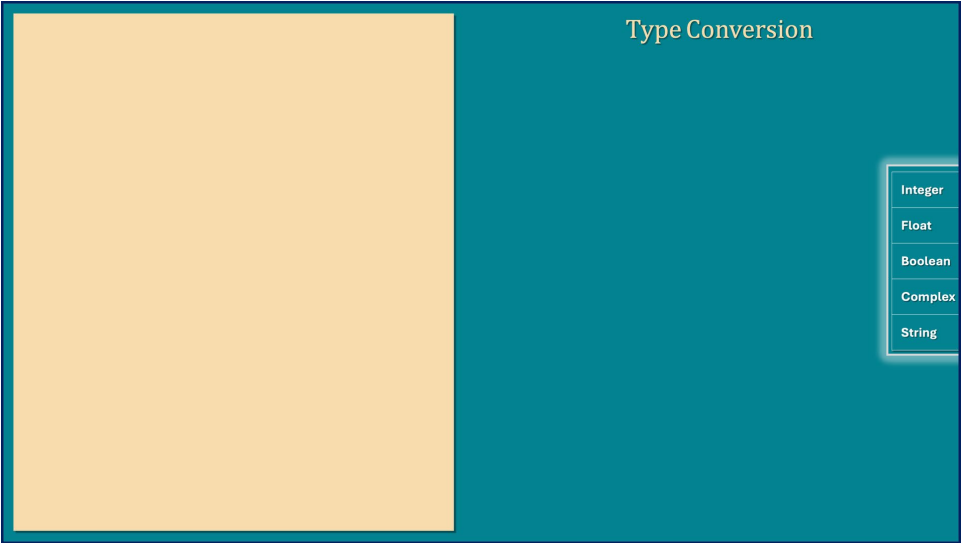
Integer

Float

Boolean

Complex

String



Type Conversion

int()

float()

bool()

complex()

str()

Integer

Float

Boolean

Complex

String

Type Conversion

int( )

float()

bool()

complex( )

str()

Integer

Float

Boolean

Complex

String

Type Conversion

int( )

Integer

Float

Boolean

Complex

String

Type Conversion

int( )

Integer

Float

Boolean

Complex

String

Type Conversion  
16.59  
int( )

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |

Type Conversion  
int( 16.59 )

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |

## Type Conversion

16 ← **int**( 16.59 )

|              |
|--------------|
| Integer      |
| <b>Float</b> |
| Boolean      |
| Complex      |
| String       |

Type Conversion

16 ← **int**( 16.59 )

Integer

**Float**

Boolean

Complex

String

Type Conversion

16 ← **int**( 16.59 )

f = 16.59

Integer

**Float**

Boolean

Complex

String



Type Conversion

16 ← int( 16.59 )

f = 16.59  
x = int( f )

Integer

**Float**

Boolean

Complex

String

Type Conversion

16 ← int( 16.59 )

f = 16.59  
x = int( f )

x

16

Integer

**Float**

Boolean

Complex

String

Type Conversion  
  
**int( )**

Integer

Float

**Boolean**

Complex

String

Type Conversion  
  
True  
  
**int( )**

Integer

Float

**Boolean**

Complex

String

Type Conversion

True  
↓  
int( )

Integer

Float

Boolean

Complex

String

Type Conversion

int( True )

Integer

Float

Boolean

Complex

String

Type Conversion

1 ← int( True )

Integer

Float

Boolean

Complex

String

Type Conversion

1 ← int( True )

Integer

Float

Boolean

Complex

String

## Type Conversion

1 ← **int**( True )

b = True

Integer  
Float  
**Boolean**  
Complex  
String

Type Conversion

1 ← int( True )

b = True  
x = int( b )

Integer

Float

**Boolean**

Complex

String

Type Conversion

1 ← int( True )

b = True  
x = int( b )

x1

Integer

Float

**Boolean**

Complex

String

Type Conversion

int( )

Integer

Float

Boolean

Complex

String

Type Conversion

int( )

Integer

Float

Boolean

Complex

String

Type Conversion

int( )

Integer

Float

Boolean

Complex

String

Type Conversion

'Alexa'

int( )

Integer

Float

Boolean

Complex

String



Type Conversion  
  
‘125’  
  
int( )

Integer

Float

Boolean

Complex

String

Type Conversion  
  
int( ‘125’ )

Integer

Float

Boolean

Complex

String

Type Conversion

125 ← int( '125' )

Integer

Float

Boolean

Complex

String

Type Conversion

125 ← int( '125' )

Integer

Float

Boolean

Complex

String

Type Conversion

125 ← **int**( '125' )

s = '125'

Integer

Float

Boolean

Complex

**String**

Type Conversion

125 ← int( '125' )

s = '125'  
x = int( s )

Integer

Float

Boolean

Complex

String

Type Conversion

125 ← int( '125' )

s = '125'  
x = int( s )

x

125

Integer

Float

Boolean

Complex

String

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

Integer

Float

Boolean

Complex

String

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

int('0b1010', 2)

Integer

Float

Boolean

Complex

String

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2 )

Integer

Float

Boolean

Complex

String

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2 )

int('0xA', 16 )

Integer

Float

Boolean

Complex

String

### Type Conversion

16 ← **int**( 16.59 )

1 ← **int**( True )

125 ← **int**( '125' )

10 ← **int**( '0b1010' , 2 )

10 ← **int**( '0xA' , 16 )

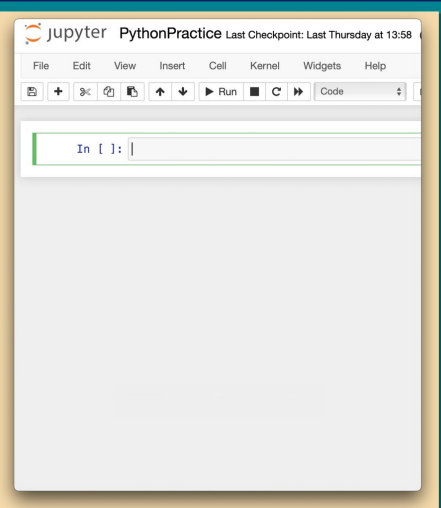
Integer

Float

Boolean

Complex

String



### Type Conversion

16 ← **int**( 16.59 )

1 ← **int**( True )

125 ← **int**( '125' )

10 ← **int**( '0b1010' , 2 )

10 ← **int**( '0xA' , 16 )

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↶

↷

↶

↷

Run

↶

↷

Code

In [ ]: f = 16.59

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2)

10 ← int('0xA', 16)

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↶

↷

↶

↷

Run

↶

↷

Code

In [ ]: f = 16.59  
b = True

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2)

10 ← int('0xA', 16)

Integer

Float

Boolean

Complex

String













Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↻

↕

↶

↷

▶ Run

■

↶

↷

Code

In [1]:

f = 16.59  
b = True  
s1 = '125'  
s2 = '0b1010'  
s3 = '0xA'

In [2]:

x = int(f)  
print(x, type(x))  
16 <class 'int'>

In [3]:

int(b)  
Out[3]: 1

In [4]:

int(s1)  
Out[4]: 125

In [ ]:

x = int(s2)

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2)

10 ← int('0xA', 16)

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↻

↕

↶

↷

▶ Run

■

↶

↷

Code

In [1]:

f = 16.59  
b = True  
s1 = '125'  
s2 = '0b1010'  
s3 = '0xA'

In [2]:

x = int(f)  
print(x, type(x))  
16 <class 'int'>

In [3]:

int(b)  
Out[3]: 1

In [4]:

int(s1)  
Out[4]: 125

In [ ]:

x = int(s2)  
print(x, type(x))

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2)

10 ← int('0xA', 16)

Integer

Float

Boolean

Complex

String

```
Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58
File Edit View Insert Cell Kernel Widgets Help
+ - + Run Code
s2 = 16.59
s3 = '0xA'

In [2]: x = int(f)
        print(x, type(x))
        16 <class 'int'>

In [3]: int(b)
Out[3]: 1

In [4]: int(s1)
Out[4]: 125

In [5]: x = int(s2)
        print(x, type(x))

ValueError
Cell In[5], line 1
----> 1 x = int(s2)
      2 print(x, type(x))

ValueError: invalid literal for int() with b
```

## Type Conversion

16 ← **int( 16.59 )**

1 ← **int( True )**

125 ← **int( '125' )**

10 ← **int('0b1010', 2)**

10 ← **int('0xA', 16)**

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |







Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

9<

📄

⬆

⬇

▶ Run

⏮

⏪

Code

In [8]:

f = 16.59  
b = True  
s1 = '125'  
s2 = '0b1010'  
s3 = '0xA'

In [9]:

int('Alexa')

ValueError  
Cell In[9], line 1  
→ 1 int('Alexa')

ValueError: invalid literal for int() with base 10

In [10]:

int(3+4j)

TypeError  
Cell In[10], line 1  
→ 1 int(3+4j)

TypeError: int() argument must be a string, a bytes-like object or a real number, not a complex number

Type Conversion

16 ← int( 16.59 )

1 ← int( True )

125 ← int( '125' )

10 ← int('0b1010', 2)

10 ← int('0xA', 16)

Integer

Float

Boolean

Complex

String

Type Conversion

✓ int()  
float()  
bool()  
complex()  
str()

Integer

Float

Boolean

Complex

String

Type Conversion

float( )

✓ int()

bool()

complex()

str()

Integer

Float

Boolean

Complex

String

Type Conversion

float( )

Integer

Float

Boolean

Complex

String

Type Conversion

float( )

✓ Integer

✓ Float

✓ Boolean

✗ Complex

✓ String

Type Conversion

float( )

Integer

Float

Boolean

Complex

String

Type Conversion

float( 125 )

Integer

Float

Boolean

Complex

String

Type Conversion

125.0 ← float( 125 )

Integer

Float

Boolean

Complex

String

Type Conversion

125.0 ← float( 125 )

float( True )

Integer

Float

Boolean

Complex

String

Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

Integer

Float

Boolean

Complex

String

Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

float( '12.75' )

Integer

Float

Boolean

Complex

String

### Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

12.75 ← float( '12.75' )

Integer

Float

Boolean

Complex

String

jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

⌕

🔍

⬆

⬇

▶ Run

⏮

⏪

Code

In [1]:

i = 125  
b = True  
s = '12.75'

In [ ]:

### Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

12.75 ← float( '12.75' )

Integer

Float

Boolean

Complex

String





Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↶↷

↶↷

↶↷

Run

↶↷

↶↷

Code

In [1]:

i = 125  
b = True  
s = '12.75'

In [2]:

x = float(i)  
print(x, type(x))  
125.0 <class 'float'>

In [ ]:

float(b)

Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

12.75 ← float('12.75')

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↶↷

↶↷

↶↷

Run

↶↷

↶↷

Code

In [1]:

i = 125  
b = True  
s = '12.75'

In [2]:

x = float(i)  
print(x, type(x))  
125.0 <class 'float'>

In [3]:

float(b)

Out[3]:

1.0

In [ ]:

Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

12.75 ← float('12.75')

Integer

Float

Boolean

Complex

String



PythonPractice

Last Checkpoint: Last Thursday at 13:58

File

Edit

View

Insert

Cell

Kernel

Widgets

Help

In [1]:

i = 125

b = True

s = '12.75'

In [2]:

x = float(i)

print(x, type(x))

125.0 <class 'float'>

In [3]:

float(b)

Out[3]: 1.0

In [4]:

float(s)

Out[4]: 12.75

In [ ]:

Type Conversion

125.0 ← float( 125 )

1.0 ← float( True )

12.75 ← float('12.75')

Integer

Float

Boolean

Complex

String

Type Conversion

✓ int()

✓ float()

bool()

complex()

str()

Integer

Float

Boolean

Complex

String

Type Conversion

bool( )

Integer

Float

Boolean

Complex

String

✓int()

✓float()

complex()

str()

Type Conversion

bool( )

Integer

Float

Boolean

Complex

String

Type Conversion

**bool( )**

✓ Integer

✓ Float

✓ Boolean

✓ Complex

✓ String

Type Conversion

**bool(anything)**

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool**(anything)

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool**(anything)

**bool**( False )

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool**(anything)

False ← **bool**( False )

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool**(anything)

False ← **bool**( False )

**bool**( 0 )

Integer

Float

Boolean

Complex

String



## Type Conversion

True ← **bool**(anything)

False ← **bool**( False )

False ← **bool**( 0 )

Integer

Float

Boolean

Complex

String

## Type Conversion

True ← **bool**(anything)

False ← **bool**( False )

False ← **bool**( 0 )

**bool**( )

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |

## Type Conversion

True ← **bool**(anything)

False ← **bool**( False )

False ← **bool**( 0 )

False ← **bool**( )

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |

Type Conversion  
**bool( 10 )**

Integer

Float

Boolean

Complex

String

Type Conversion  
True ← **bool( 10 )**

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool( 10 )**

**bool(-12 )**

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool( 10 )**

True ← **bool(-12 )**

Integer

Float

Boolean

Complex

String

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

**bool( -1.2E-3 )**

Integer

**Float**

Boolean

Complex

String

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

Integer

**Float**

Boolean

Complex

String

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

**bool( 3+4j )**

Integer

Float

Boolean

**Complex**

String

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

True ← **bool( 3+4j )**

Integer

Float

Boolean

**Complex**

String

## Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool(-1.2E-3 )**

True ← **bool( 3+4j )**

**bool( 'False' )**

|               |
|---------------|
| Integer       |
| Float         |
| Boolean       |
| Complex       |
| <b>String</b> |

## Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool**(-1.2E-3 )

True ← `bool( 3+4j )`

True ← **bool( 'False' )**

- Integer
- Float
- Boolean
- Complex
- String

## Type Conversion

True ← **bool( 10 )**

True ← `bool( -12 )`

True ← `bool(-1.2E-3)`

True ← `bool( 3+4j )`

True ← **bool( 'False' )**

- Integer
- Float
- Boolean
- Complex
- String





Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↺

↻

▶ Run

⏮

⏪

⏩

⏭

Code

In [1]:

i1 = 10  
i2 = -12  
f = -1.2e-3  
c = 3+4j  
s = 'False'

In [2]:

x = bool(i1)  
print(x, type(x))  
True <class 'bool'>

In [ ]:

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↺

↻

▶ Run

⏮

⏪

⏩

⏭

Code

In [1]:

i1 = 10  
i2 = -12  
f = -1.2e-3  
c = 3+4j  
s = 'False'

In [2]:

x = bool(i1)  
print(x, type(x))  
True <class 'bool'>

In [3]:

bool(i2)

Out[3]:

True

In [ ]:

Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

Integer

Float

Boolean

Complex

String



Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↻

⬆

⬇

⬆

⬇

▶ Run

■

↺

↻

▶ Code

```
1 x = 11
2 c = 3+4j
3 s = 'False'

In [2]: x = bool(11)
        print(x, type(x))
        True <class 'bool'>

In [3]: bool(12)
Out[3]: True

In [4]: bool(f)
Out[4]: True

In [5]: bool(c)
Out[5]: True

In [ ]: bool(s)
```

Type Conversion

True ← **bool( 10 )**

True ← **bool(-12 )**

True ← **bool(-1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↻

⬆

⬇

⬆

⬇

▶ Run

■

↺

↻

▶ Code

```
In [2]: x = bool(11)
        print(x, type(x))
        True <class 'bool'>

In [3]: bool(12)
Out[3]: True

In [4]: bool(f)
Out[4]: True

In [5]: bool(c)
Out[5]: True

In [6]: bool(s)
Out[6]: True

In [ ]:
```

Type Conversion

True ← **bool( 10 )**

True ← **bool(-12 )**

True ← **bool(-1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

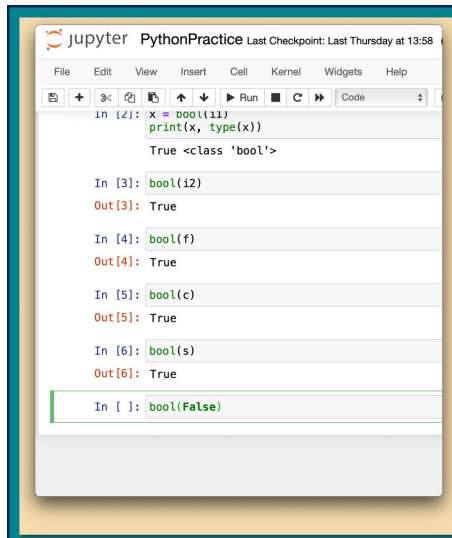
Integer

Float

Boolean

Complex

String



A screenshot of a Jupyter Notebook interface. The title bar shows 'jupyter PythonPractice' and 'Last Checkpoint: Last Thursday at 13:58'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for file operations, code execution, and output viewing. The code area shows the following cells:

```
In [2]: x = bool(11)
        print(x, type(x))
        True <class 'bool'>
```

```
In [3]: bool(12)
Out[3]: True
```

```
In [4]: bool(f)
Out[4]: True
```

```
In [5]: bool(c)
Out[5]: True
```

```
In [6]: bool(s)
Out[6]: True
```

```
In [ ]: bool(False)
```

## Type Conversion

True ← **bool( 10 )**

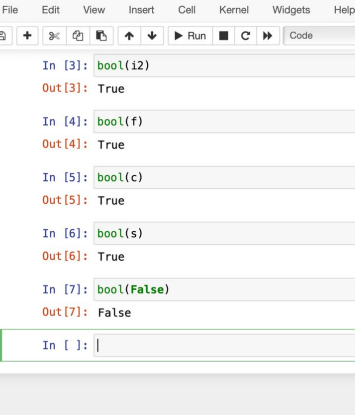
True ← **bool( -12 )**

True ← **bool(-1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

|         |
|---------|
| Integer |
| Float   |
| Boolean |
| Complex |
| String  |



The screenshot displays a Jupyter Notebook window titled "jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58". The interface includes a top menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Below the menu is a toolbar with icons for file operations (new, open, save, print), code execution (run, step, interrupt), and code management (code, cell, notebook). The main area shows a series of input-output pairs:

```
In [3]: bool(12)
Out[3]: True

In [4]: bool(f)
Out[4]: True

In [5]: bool(c)
Out[5]: True

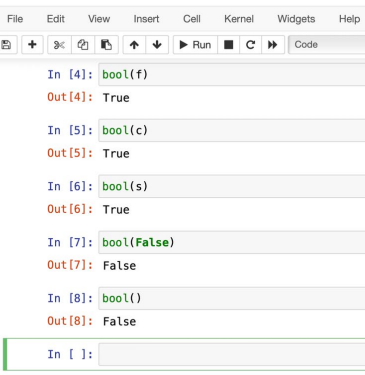
In [6]: bool(s)
Out[6]: True

In [7]: bool(False)
Out[7]: False

In [ ]: 
```

The notebook cells are styled with alternating light gray and white backgrounds. The output of each cell is displayed below the input line, with the prompt "Out[ ]:" indicating the result of the execution.

```
True ← bool( 10 )
True ← bool( -12 )
True ← bool( -1.2E-3 )
True ← bool( 3+4j )
True ← bool( 'False' )
```



Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

Run Code

```
In [4]: bool(f)
Out[4]: True

In [5]: bool(c)
Out[5]: True

In [6]: bool(s)
Out[6]: True

In [7]: bool(False)
Out[7]: False

In [8]: bool()
Out[8]: False

In [ ]:
```

## Type Conversion

True ← **bool( 10 )**

True ← **bool( -12 )**

True ← **bool( -1.2E-3 )**

True ← **bool( 3+4j )**

True ← **bool( 'False' )**

Type Conversion

Integer

Float

Boolean

Complex

String

✓ **int()**

✓ **float()**

✓ **bool()**

**complex()**

**str()**

Type Conversion

**complex( )**

Integer

Float

Boolean

Complex

String

✓ **int()**

✓ **float()**

✓ **bool()**

**str()**

Type Conversion

**complex( )**

Integer

Float

Boolean

Complex

String

Type Conversion

**complex( )**

✓ Integer

✓ Float

✓ Boolean

✓ Complex

✓ String



Type Conversion  
**complex( 10 )**

Integer

Float

Boolean

Complex

String

Type Conversion  
**10+0j ← complex( 10 )**

Integer

Float

Boolean

Complex

String

Type Conversion  
 $10+0j \leftarrow \text{complex}(10)$   
 $\text{complex}(-12.5)$ 

Integer

Float

Boolean

Complex

String

Type Conversion  
 $10+0j \leftarrow \text{complex}(10)$   
 $-12.5+0j \leftarrow \text{complex}(-12.5)$ 

Integer

Float

Boolean

Complex

String

## Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$\text{complex}(\text{True})$

|                |
|----------------|
| Integer        |
| Float          |
| <b>Boolean</b> |
| Complex        |
| String         |

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

Integer

Float

Boolean

Complex

String

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

$\text{complex}(\text{False})$

Integer

Float

Boolean

Complex

String

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

$0j \leftarrow \text{complex}(\text{False})$

Integer

Float

Boolean

Complex

String

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

$0j \leftarrow \text{complex}(\text{False})$

$\text{complex}('3+4j')$

Integer

Float

Boolean

Complex

String

### Type Conversion

$10+0j \leftarrow \text{complex}( 10 )$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}( \text{True} )$

$0j \leftarrow \text{complex}( \text{False} )$

$3+4j \leftarrow \text{complex}('3+4j')$

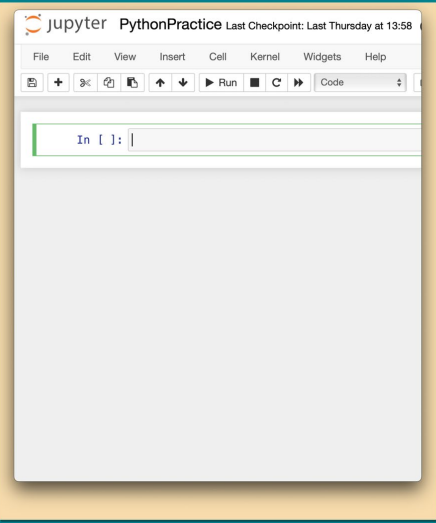
Integer

Float

Boolean

Complex

String



### Type Conversion

$10+0j \leftarrow \text{complex}( 10 )$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}( \text{True} )$

$0j \leftarrow \text{complex}( \text{False} )$

$3+4j \leftarrow \text{complex}('3+4j')$

Integer

Float

Boolean

Complex

String



Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

9<

Code

```
In [1]: i = 10
        f = -12.5
        b1 = True
        b2 = False
        s = '3+4j'

In [2]: x = complex(i)
        print(x, type(x))
        (10+0j) <class 'complex'>

In [ ]:
```

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

$0j \leftarrow \text{complex}(\text{False})$

$3+4j \leftarrow \text{complex}('3+4j')$

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

9<

Code

```
In [1]: i = 10
        f = -12.5
        b1 = True
        b2 = False
        s = '3+4j'

In [2]: x = complex(i)
        print(x, type(x))
        (10+0j) <class 'complex'>

In [ ]: complex(f)
```

Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

$0j \leftarrow \text{complex}(\text{False})$

$3+4j \leftarrow \text{complex}('3+4j')$

Integer

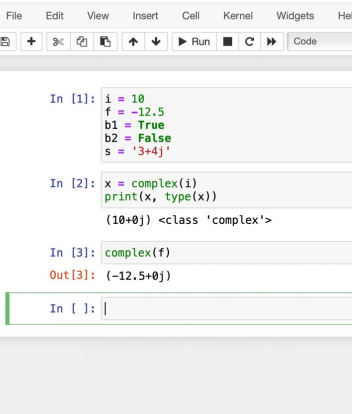
Float

Boolean

Complex

String





The screenshot shows the Jupyter PythonPractice web interface. At the top, the title bar reads "Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58". Below the title bar is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Under the "Cell" menu, there are icons for code, text, and raw, along with buttons for "Run", "C" (Copy), and "P" (Paste). A "Code" button and a "Go" button are also visible. The main content area displays a code cell with the following code:

```
In [1]: i = 10
        f = -12.5
        b1 = True
        b2 = False
        s = '3+4j'
```

```
In [2]: x = complex(i)
        print(x, type(x))

        (10+0j) <class 'complex'>
```

```
In [3]: complex(f)

Out[3]: (-12.5+0j)
```

The code cell is currently selected, and the input prompt "In [ ]: " is visible at the bottom of the cell.

## Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

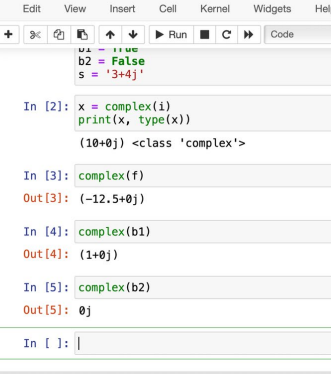
**1+0j ← complex( True )**

0j ← **complex( False )**

$3+4j \leftarrow \text{complex}('3+4j')$

- Integer
- Float
- Boolean
- Complex
- String**





Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

```
a1 = 1+2j
b2 = False
s = '3+4j'
```

In [2]: `x = complex(i)`  
`print(x, type(x))`  
`(10+0j) <class 'complex'>`

In [3]: `complex(f)`  
`Out[3]: (-12.5+0j)`

In [4]: `complex(b1)`  
`Out[4]: (1+0j)`

In [5]: `complex(b2)`  
`Out[5]: 0j`

In [ ]: |

## Type Conversion

$10+0j \leftarrow \text{complex}(10)$

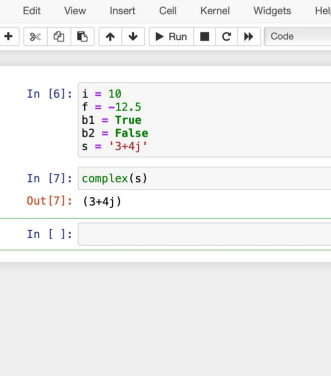
$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j \leftarrow \text{complex}(\text{True})$

```
0j ← complex( False )
```

$3+4j \leftarrow \text{complex}('3+4j')$

- Integer
- Float
- Boolean
- Complex
- String**



The screenshot displays the JupyterLab application window. At the top, the title bar reads "Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58". Below the title bar is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. A toolbar is located beneath the menu bar, containing icons for file operations (new, open, save, close), editing (undo, redo), and execution (run, step, interrupt, restart, clear output, copy, paste). The main workspace is divided into two panes. The left pane is a code editor showing a Python script with the following code:

```
In [6]: i = 10
        f = -12.5
        b1 = True
        b2 = False
        s = '3+4j'
```

The right pane is a command prompt showing the output of the previous cell:

```
In [7]: complex(s)
Out[7]: (3+4j)
```

Below the command prompt, there is an input line for the next command, which currently contains "In [ ]:" followed by a text input field.

## Type Conversion

$10+0j \leftarrow \text{complex}(10)$

$-12.5+0j \leftarrow \text{complex}(-12.5)$

$1+0j$  ← `complex( True )`

```
0j ← complex( False )
```

$3+4j \leftarrow \text{complex}('3+4j')$

- Integer
- Float
- Boolean
- Complex
- String

Type Conversion

Integer

Float

Boolean

Complex

String

✓ **int()**

✓ **float()**

✓ **bool()**

✓ **complex()**

**str()**

Type Conversion

Integer

Float

Boolean

Complex

String

✓ **int()**

✓ **float()**

✓ **bool()**

✓ **complex()**

**str( )**

Type Conversion

str( )

Integer

Float

Boolean

Complex

String

Type Conversion

str( )

✓ Integer

✓ Float

✓ Boolean

✓ Complex

✓ String

Type Conversion

**str( anything )**

Integer

Float

Boolean

Complex

String

Type Conversion

**str ← str( anything )**

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

str( -12 )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

Integer

Float

Boolean

Complex

String



Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

str( -1.2E-3 )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

str( False )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

'False' ← str( False )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

'False' ← str( False )

str( 3+4j )

Integer

Float

Boolean

Complex

String

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

'False' ← str( False )

'3+4j' ← str( 3+4j )

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↺

↻

▶ Run

■

↺

↻

Code

In [1]:

```
i1 = 10
i2 = -12
f = -1.2E3
b1 = False
c = 3+4j
```

In [ ]:

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

'False' ← str( False )

'3+4j' ← str( 3+4j )

Integer

Float

Boolean

Complex

String

Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58

File Edit View Insert Cell Kernel Widgets Help

+

↶

↷

↺

↻

▶ Run

■

↺

↻

Code

In [1]:

```
i1 = 10
i2 = -12
f = -1.2E3
b1 = False
c = 3+4j
```

In [2]:

```
x = str(i1)
print(x , type(x))
```

In [ ]:

Type Conversion

'10' ← str( 10 )

'-12' ← str( -12 )

'-0.0012' ← str( -1.2E-3 )

'False' ← str( False )

'3+4j' ← str( 3+4j )

Integer

Float

Boolean

Complex

String

The screenshot shows a Jupyter Notebook window titled "Jupyter PythonPractice Last Checkpoint: Last Thursday at 13:58". The interface includes a top menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Below the menu is a toolbar with icons for file operations, cell navigation, and execution. The main area displays a code cell with the following Python code:

```
In [1]: i1 = 10
        i2 = -12
        f = -1.2E3
        b1 = False
        c = 3+4j
```

Below the code cell, the output of the previous cell is shown:

```
Out [2]: x = str(i1)
         print(x, type(x))
         10 <class 'str'>
```

The next code cell contains:

```
In [3]: str(i2)
```

Its output is:

```
Out [3]: '-12'
```

The following code cell contains:

```
In [4]: str(f)
```

Its output is:

```
Out [4]: '-1200.0'
```

The final code cell shows the prompt:

```
In [ ]: |
```

## Type Conversion

**'10' ← str( 10 )**

**'-12' ← str( -12 )**

```
'-0.0012' ← str(-1.2E-3)
```

**'False' ← str( False )**

`'3+4j' ← str( 3+4j )`

- Integer
- Float
- Boolean
- Complex**
- String

